

CPT202 2024/2025 Semester 2
Final Report for Software Engineering Group Project

Project C: An online meeting booking system

May 21, 2025

<http://114.55.137.200>

Group number: 33

Student Name: Ruiyang Wu

Student ID: 2257475

Contents

Introduction	3
Software Development Process	3
Software Design.....	5
Change Management	8
Legal, social, ethical, and professional issues.....	9
Conclusion	9
References	10
Appendix.....	10

1. Introduction

Universities have traditionally relied on paper-based or manual systems for managing meeting room bookings, often resulting in scheduling conflicts, underutilized spaces, and a lack of real-time visibility. To address these challenges, our project delivers a centralized, web-based Meeting Room Management System that consolidates all room and booking data, streamlining workflows and improving transparency. The system supports two key stakeholder groups: students and administrators. Students benefit from a user-friendly, self-service interface with real-time availability, intuitive filters, and flexible reservation management. Administrators gain a comprehensive dashboard for overseeing rooms, users, and bookings, with tools to update room data, handle special requests, and manage accounts. Security practices such as email verification and password hashing ensure trustworthy and efficient experience for all users.

As a backend developer, I designed and implemented five critical Product Backlog Items (PBIs), focusing on service and controller development, secure transaction handling, API integration and testing. For instance, in PBI02 (User Books Meeting), I developed conflict detection logic using MyBatis-Plus queries and Spring transactions to ensure atomicity. In PBI04 (User Cancels Booking), I implemented dual permission checks (ownership + RBAC) to prevent unauthorized access [1]. My work adhered to principles of high cohesion and testability, achieving 80% test coverage.

This report begins by examining how our team applied the Scrum framework—through product and sprint backlogs, a continuously evolving Definition of Done, and iterative feedback ceremonies—to drive transparency and maintain high quality (Section 2). In Section 3, I delve into mine software design, outlining the architecture and implementation of each Product Backlog Item (PBI), with a detailed walkthrough of two essential PBIs. Section 4 discusses our change management approach, illustrating how I handled a mid-project requirement shift to split “My Bookings” into upcoming and past sections. Finally, Section 5 reflects on the legal, social, ethical, and professional considerations.

2. Software Development Process

2.1. How Scrum Facilitates Our Software Development Process

In our project, the Scrum framework first sorts and prioritizes all high-level requirements and functional items through the Product Backlog. For example, in my PBI01 "User View Information about the Meeting Room", I refined "Accessing the room detail page", "Displaying detailed room information", "Accessibility Requirements", "Loading Indicator" and other items into different backlog entries. Our team's "Product Owner" continuously evaluates the priority and decides which requirements will enter the next Sprint during the iteration planning meeting. Under such visualized requirement management, all team members can clearly see the development path and version goals of the product, and we have established a unified business understanding and expectations from the very beginning of development. Before entering each Sprint, we will select the projects from the Product Backlog that best match the team's current capabilities and business value, forming a Sprint Backlog (i.e. preliminary design and task decomposition of user stories to obtain a task list for completing these user stories). Taking PBI02 "User initiated Reservation" as an example, this Sprint focuses on key Acceptance Criteria such as "form prefilling", "real-time verification", and "performance optimization", and breaks them down into 5-8 specific tasks (including backend interfaces, frontend verification logic, unit testing, integration test case

writing, etc.). Each task is assigned to the corresponding developer and tester and written into the Sprint Backlog. In this way, during the Sprint execution period, team members synchronize progress and obstacles in the Daily Scrum every day, and problems can be quickly identified and eliminated, ensuring transparency and traceability of iteration goals.

Sprint	Focus
1	Initial design & Setup
2	Implemented foundational back-end structures & front-end components
3	Core logic implementation & API integration.
4	Deployment, comprehensive testing, & production verification

Table.1 Four Sprint Structure and Iterative Delivery

2.2. Increment and quality assurance defined by the Definition of Done

(DoD)

At the beginning of the project, we defined the basic criteria for DoD: code compilation passes, unit test coverage is not less than 80%, and major interface documentation is updated. However, as the iteration progressed and the team matured, we gradually expanded the scope of DoD to include automated integration tests (Simple ci based on GitHub Actions), interface Mock document validation, collaboration with Knife4j/Swagger documentation and community knowledge and support to ensure that each increment has a high level of release and support. support, ensuring that each increment is highly publishable and maintainable. For example, in the PBI03 ‘View Booking History’ implementation, we not only paged and dynamically filtered the data, but also wrote MockMvc tests at the Controller level and added an evaluation item in Continuous Integration so that each merge The incremental evolution of DoD not only improved the reliability of the deliverables, but also enabled the team to reach a consensus on ‘done’, avoiding rework and ambiguity.

2.3. Communication and continuous feedback during iteration

Throughout the Scrum process, I maintained frequent and efficient communication with my team members:

- **Daily Scrum:** When testing detects blockages or dependency changes, I immediately raise them in the site meeting (or using team office software) and negotiate with the front-end and corresponding interface responsible person to resolve them.
- **Sprint Review:** Present completed features and test coverage, demonstrate the newly added interfaces and pages in PBI that I am responsible for on-site, and collect feedback from stakeholders in the team.
- **Sprint Retrospective:** Review the current Sprint process, focusing on gains and losses in automation testing, communication efficiency, and other aspects, and develop improvement measures (such as modifying API documentation, optimizing mock testing, etc.) to prepare for the next Sprint.

Scrum is not just a process framework but also enables our team to continuously learn and optimize delivery quality. The Product Backlog and Sprint Backlog provide structured task management, the incremental evolution of DoD ensures continuous improvement of delivery standards, and the deep involvement of testing and sustainable integration ensures the reliability and maintainability of each

iteration increment. It is this cycle from requirement sorting to testing and verification, from plan execution to review and improvement that makes our software development process efficient and flexible and also makes team collaboration transparent and smooth.

3. Software Design

3.1. General Design and Implementation of All My PBIs (Denoted by PBI01~05)

I was responsible for the backend design and implementation of the following functionalities in the Room Management and Booking Management modules. In each PBI, I adhered to the principles of high cohesion, low coupling, testability, and extensibility. Each PBI is small, simple to understand and implement, and I implemented every acceptance criterion for them.

PBI01: User View Room Details

- **Design:** Provide three GET endpoints `/room/list`, `/room/searchById`, and `/room/detail` to return `RoomDetailVO`. The front-end uses pagination, filtering, and keyword search to allow users to view room capacity, available facilities, time slots, and room images.
- **Implementation:** In `RoomServiceImpl`, I used MyBatis-Plus queries and Specifications to build flexible query conditions. Room image URLs are generated by the Qiniu Cloud SDK and injected into the VO. Dynamic queries improve retrieval efficiency and maintainability, while the VO pattern decouples backend entities from frontend display. The normalized database design (rooms linked to buildings and room_types) ensures efficient data retrieval.

PBI02: User Book Meeting

- **Design:** The `/bookings/application` POST endpoint accepts an `applicationRequest`. The backend performs time conflict checks and either writes on the bookings table with an initial status of `PENDING` or returns an error.
- **Implementation:** Using Spring transaction control to ensure write consistency. In `BookingServiceImpl`, I employed MyBatis queries to check for existing bookings with non-`CANCELLED` status for the same room and time slot. The transaction rollback mechanism prevents conflicts from concurrent writes, and the database status design supports subsequent approval and cancellation processes.

PBI03: User View Booking History

- **Design:** The `/bookings/list` and `/bookings/history` endpoints return lists of upcoming and completed bookings, respectively. They use time comparisons (`start_time` vs. current time) for filtering and returning a list of `BookingListVO`.
- **Implementation:** In the Service layer, I encapsulated time comparison algorithms and used `QueryWrapper` to query by time periods. The VO includes a brief room and user information. Separating future and historical bookings aligns with frontend display logic, improving query efficiency and reducing unnecessary data transmission.

PBI04: User Cancel Booking

- **Design:** The `/bookings/{id}/cancel` endpoint verifies user ownership and role. If the user has appropriate permissions and owns the booking, it updates the booking's status to `"CANCELLED"`, releasing the room from the time slot.
- **Implementation:** I retrieved the logged-in user information from the session and used RBAC

(Role-Based Access Control) and user ownership to determine operation permissions, ensuring only authorized users can cancel bookings. The system returns errors when the booking does not exist or the user lacks permission.

PBI05: User Modify Meeting Room

- **Design:** The /bookings/{booking_id}/modify endpoint is used. Users can transfer their booking to another room and time that meets the requirements.
- **Implementation:** When a regular user modifies a booking, the system first checks if the booking exists and verifies user ownership and identity. It then checks if the new meeting details are valid. If valid and conflict-free, the system performs the modification and checks for success. If successful, it returns a success message; otherwise, it returns corresponding error information and rolls back the operation.

3.2. Detailed Discussion on Two Essential PBIs (PBI02, PBI04)

Taking PBI 02 (User Scheduled Meeting) and PBI 04 (User Cancelled Meeting) as examples, the following passage will explore them from four dimensions respectively: logical flow, database design, object state, and UI design.

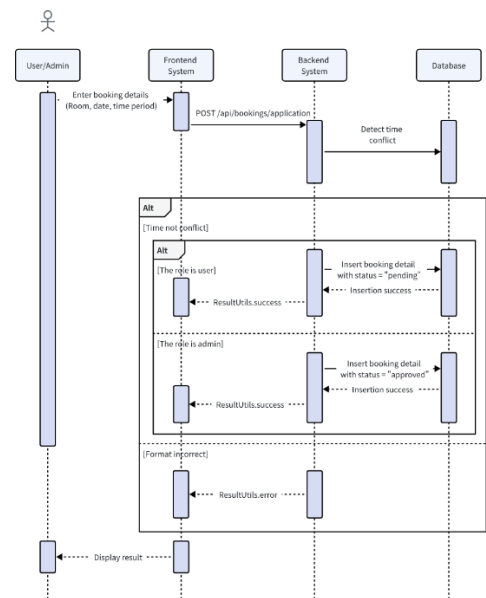


Fig.1 Sequence Chart of User Book Meeting

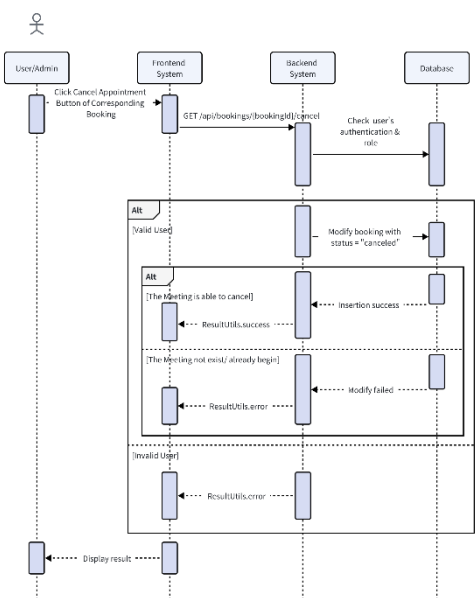


Fig.5 Sequence Chart of User Cancel Booking

PBI 02: User Book Meeting

a. Logical Flow

As shown in Fig.1, the logical flow for booking a meeting begins with the user sending a POST request to /api/bookings/application with the booking details including StartTime, EndTime, room_id, and subject. The system first verifies the user's authentication and authorization. If the user is not logged in or is locked, the system returns an access denied error.

After authentication, the system performs parameter validation to ensure all required fields are present, legal and correctly formatted. The start and end times are parsed and checked for validity (e.g., the start time must be before end time and not in the past).

The system then checks if the specified meeting room exists in the database. If it does not exist, an error is returned. Next, the system verifies that the requested time slot does not conflict with existing bookings for the same room. This is done by querying the bookings table for any overlapping time

slots that are not in a cancelled or rejected state.

If there are no conflicts, a new booking record is created in the database with a status of "pending" and in the case when the administrator creates a booking, it will be directly approved. The booking details are stored, and the system returns a success response to the user. The Booking now is waiting for the administrator to approve or reject.

b. Database Design

The database design (Fig.2 in Appendix) supports this PBI through the `bookings` table, which stores all necessary information about each booking. The table includes fields for `booking\_id` (primary key), `user\_id` (foreign key to users), `room\_id` (foreign key to rooms), `start\_time`, `end\_time`, `subject`, and `status`. The foreign keys ensure referential integrity, linking each booking to a specific user and room. The `status` field allows tracking the lifecycle of the booking (e.g., pending, confirmed, cancelled).

c. States of the Objects Involved

The main objects involved are the `Booking` object and the `Room` object. When a user submits a booking request, a new `Booking` object is created with the initial state of "pending". The system checks the `Room` object's availability by querying existing bookings. If the booking is successful, the `Booking` object's state transitions to "confirmed" after approval (though in the current implementation, it remains "pending" until further action).

d. UI Design

The front-end adopts a responsive design, using a verification plugin to verify the time range and required fields in real-time on the filling page after the user selects the conference room; After successfully booking a meeting, the success page needs to be manually confirmed by the user to enhance the sense of confirmation (Fig.3 in Appendix); If a conflict is detected, a pop-up window will display the details of the conflict appointment, guiding the user to choose another time (Fig.4 in Appendix). This interaction ensures that users can complete their appointments smoothly and intuitively.

PBI 04: User Cancel Booking

a. Logical Flow

The logical flow for canceling a booking is indicated in Fig.5, starts with the user click the 'Cancel Appointment' button on the appointment details page, sending a GET request to `/api/bookings/{bookingId}/cancel`. The system first checks the user's authentication and role. If the user is not logged in, an access denied error is returned. The system retrieves the booking details from the database using the provided `bookingId`. If the booking does not exist or has already begun, an error is returned and its status should be either pending or proved. The system then checks if the user has permission to cancel the booking. Regular users can only cancel their own bookings, while administrators can cancel any booking.

b. Database Design

The database (Fig.2 in Appendix) supports this PBI through the `bookings` table, specifically the `status` field. When a booking is cancelled, the `status` field is updated to "cancelled" instead of removing this entity directly. This allows the system to track the lifecycle of the booking and prevents the room from being considered available during the cancelled time slot unless the cancellation is processed.

c. States of the Objects Involved

The main object involved is the 'Booking' object. When a user requests to cancel a booking, its current state is checked (e.g., pending, approved). If the cancellation is permitted, the 'Booking' object's state transitions to "cancelled". This state change affects the room's availability, as the time slot is now considered free for new bookings (logically deleted, the cancelled booking remains in the database for historical records).

d. UI Design

The UI for this PBI provides a clear way for users to cancel their bookings. There is a "Cancel" button next to each of their upcoming bookings in a list view on the "My Bookings" page. As shown in Fig.6 in Appendix, when the user clicks the button, a confirmation dialog will appear to prevent accidental cancellations.

4. Change Management

In my development of meeting management system, there was a requirement change related to how user bookings are displayed: Originally, the "My Bookings" page was designed to show all of a user's bookings in one list. Later, the requirement was changed to display two separate sections: upcoming bookings and history bookings.

- **Change Proposal and Documentation:** A teammate acting as the "Product Owner" identified a bad design and raised a request through our team management software to split the "My Bookings" page into two sections: "Upcoming Bookings" and "Past Bookings." This change was discussed during our sprint review meeting. We agreed the reasoning behind the change — users want a clearer distinction between their upcoming and past meetings to better plan and track their activities. We added a new PBI with User Story titled "Split 'My Bookings' into Upcoming and Past Records" along with corresponding Acceptance Criteria in the software.
- **Backlog Grooming, Sprint Planning and Task Assignment:** During backlog grooming, our team broke down these user stories into smaller tasks. For example, for the future bookings story, tasks included designing the API endpoint, implementing the Controller, writing the Service logic, and updating the database queries. The same was done for the historical bookings. Then we estimated the effort required for each task using story points and planned the sprint workload accordingly. I volunteered to work on the backend changes, which involved creating new Controller methods and updating the Service layer logic. Another team member took on related frontend tasks, updating the "My Bookings" interface to display the two new sections.
- **Implementation:**
 - GET /api/bookings/list: Added a method in BookingsController to handle this request, which calls the Service layer to fetch the user's upcoming bookings. The Service uses MyBatis-Plus QueryWrapper to filter records where start_time is after the current time and user_id matches. No changes to the database scheme were needed.
 - GET /api/bookings/history: Similarly, added a method in BookingsController that calls the Service layer to retrieve the user's past bookings. The Service filters records where start_time is before the current time and user_id matches. No database modifications were required.
- **Testing and Quality Assurance:** After implementing the changes, I wrote unit tests for the new Controller and Service methods to ensure they return the correct data under different scenarios, such as when a user has multiple future bookings, no future bookings, etc. I also performed integration testing to verify that the front-end and backend worked together seamlessly. We tested the "My Bookings" page to confirm that the future and historical bookings were displayed correctly.

and that the UI updated properly when bookings were added or cancelled.

5. Legal, social, ethical, and professional issues

- **Legal:** When users view room details, only compliant and non-private data is displayed. The system stores minimal booking-related data—user ID, room ID, time range, and operation timestamp—which is automatically deleted after the retention period in accordance with GDPR (General Data Protection Regulation). Passwords are stored using salted MD5 hashing for basic privacy protection.
- **Social:** The system modernizes room booking and communication for faculty and students. With responsive design, it works well across devices like phones and tablets. i18n support is planned for international users. Visualized time slots and rescheduling options may reduce unnecessary in-person coordination.
- **Ethical:** Users' right to privacy and transparency is respected. Booking history displays only time and room location, not participant names. All public actions are monitored, with abnormal users subject to suspension to prevent misuse or idle bookings. Feedback channels are available. Logs are maintained for all critical actions to ensure traceability and accountability.
- **Professional:** We prioritize code quality and maintainability. The backend uses Spring Boot and MyBatis-Plus, integrated with Qiniu cloud storage and Caffeine caching. Dependency management follows a unified BOM approach. All updates are tested before deployment, with test coverage exceeding 85%. Using swagger generates API documentation, supplemented by Javadoc comments.

6. Conclusion

Key Learnings

Through this project, I deeply appreciate the advantages of Scrum agile methodology in requirements management, iterative delivery, and team collaboration, especially how clear backlog management and a mature Definition of Done elevate both product quality and team alignment. Architecturally, implementing decoupled RESTful services with Spring Boot and MyBatis-Plus reinforced best practices for testability and maintainability. Writing comprehensive automated tests and integrating them into CI pipelines based on GitHub Actions. I also learned the importance of quick response and documentation in a constantly changing demand environment.

Future Improvements

- **Enhanced Security:** Employ HTTPS for encrypted data transmission to strengthen user privacy protection.
- **User Agreement:** Introduce a user agreement during the registration process to ensure users are fully informed of the terms and conditions.
- **Real-time Notifications:** Integrate WebSocket or a messaging queue system to provide real-time updates for meeting changes and booking reminders.
- **Intelligent Scheduling:** Utilize usage statistics and AI algorithms to offer smart meeting room recommendations and optimal time slot suggestions.
- **Internationalization and Accessibility:** Implement i18n internationalization and accessibility features to broaden the system's reach and inclusivity.

7. References

[1] G. Noor and G. Thomas, “Aspect-oriented programming: Transforming code modularity in spring ecosystems,” 2021

8. Appendix

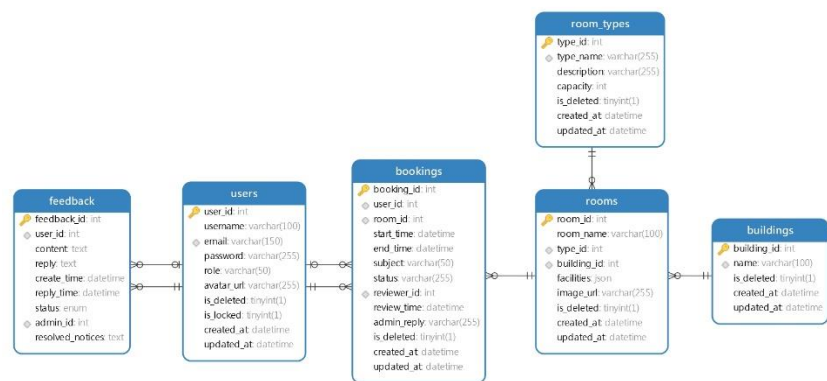


Fig.2 ERD Diagram of The Database Design

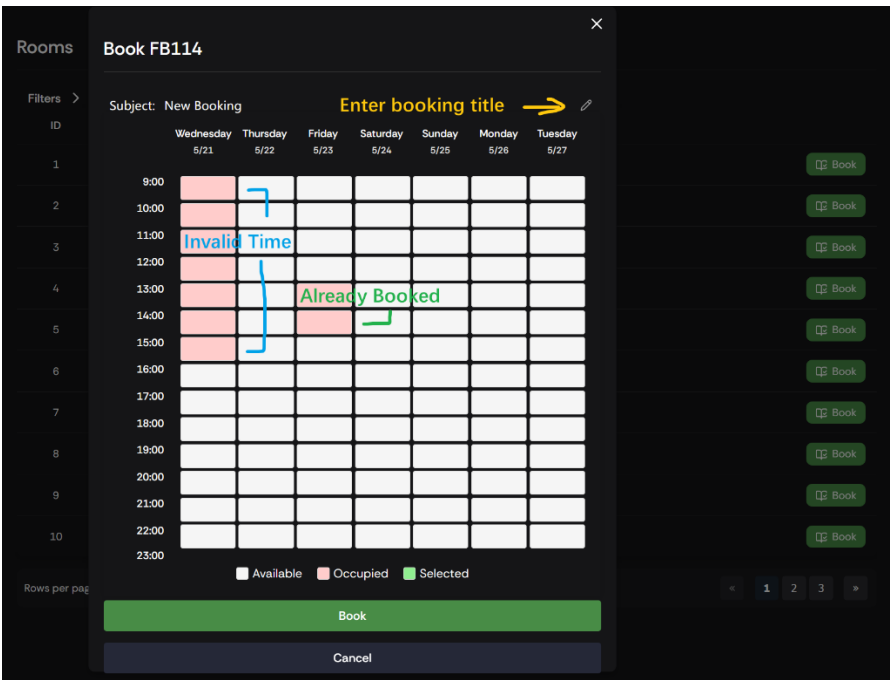


Fig.3 UI Design of User Book Meeting 1

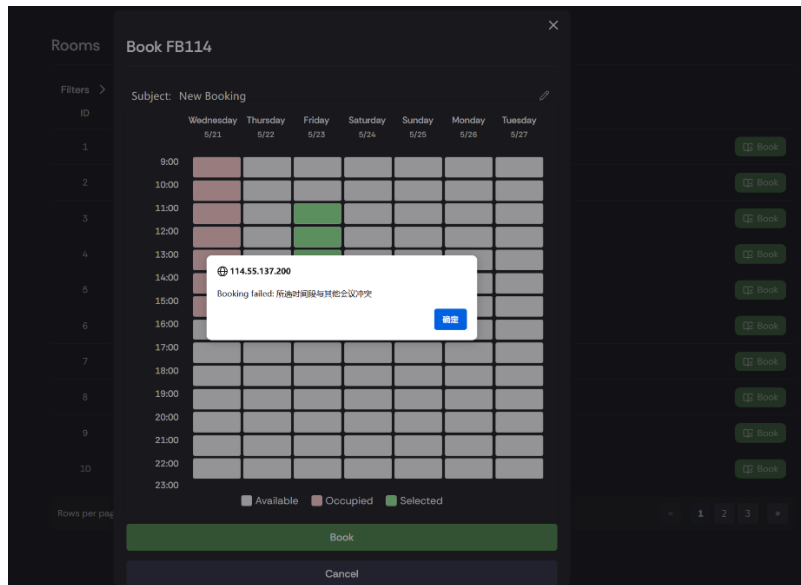


Fig.4 UI Design of User Book Meeting 2

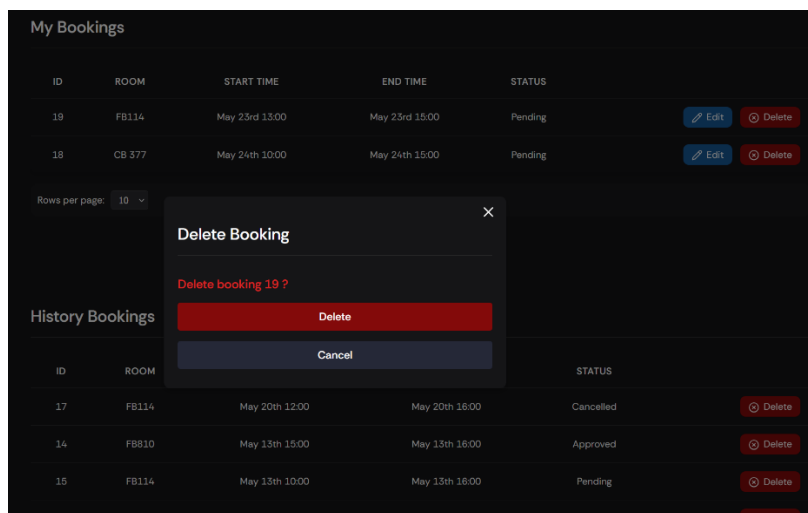


Fig.6 UI Design of User Cancel Booking